MindPal — Goal-Based Software Measurement Framework

1. BUSINESS GOAL IDENTIFICATION

The primary business goal that drove MindPal's development was defined as follows using the GQ(I)M template:

Purpose: To evaluate and deliver AI-powered therapeutic support in order to improve accessibility of mental health care for users experiencing emotional distress.

Perspective: Examine the effectiveness, reliability, and usability of the system from the viewpoint of the end user (someone in emotional crisis) and the developer (ensuring clinical accuracy and system stability).

Environment: A Node.js web application powered by the Groq API, deployed on personal or shared hosting, accessed via browser, requiring no technical knowledge from the user, targeting users who cannot easily access licensed therapists.

From this primary goal, three subgoals were derived mirroring HP's GQM structure:

Subgoal A: Maximise therapeutic usefulness and user satisfaction. Subgoal B: Minimise system failures and engineering complexity. Subgoal C: Minimise clinical inaccuracy and harmful responses.

2. ENTITY-QUESTION CHECKLIST

Following the GQ(I)M process, the following entities were identified and questions were asked about each.

Inputs and Resources

Entity: Groq API and Llama 3.3 70B model. Questions asked: Is the model capable of generating clinically appropriate therapeutic responses? Is the API free and reliable enough for sustained use? How does model temperature affect response consistency?

Entity: System prompts (therapy mode instructions). Questions asked: Are the prompts grounded in peer-reviewed clinical frameworks? Do they produce meaningfully different responses across therapy modes? Are they specific enough to guide the model toward therapeutic behaviour?

Entity: Developer (single-person project). Questions asked: Is the codebase maintainable by one person? Is the architecture simple enough to extend without breaking existing functionality?

Products and By-products

Entity: AI therapy responses. Questions asked: Are responses warm, empathetic, and clinically appropriate? Do responses vary meaningfully between modes? Does the system avoid harmful generic advice?

Entity: Session conversation history. Questions asked: Does the system maintain context across a full session? Is the history correctly passed to the API on every request?

Internal Artifacts

Entity: server.js backend. Questions asked: Is the API key securely hidden from the frontend? Are all error states handled? Does the server restart cleanly after a configuration change?

Entity: .env configuration file. Questions asked: Is the key loading correctly on startup? Is the file excluded from any version control upload?

Activities and Flowpaths

Entity: User message submission. Questions asked: Does the message reach the server correctly? Is the conversation history appended in the right order? Is the response displayed without delay or visual glitch?

Entity: Therapy mode switching. Questions asked: Does switching modes correctly change the system prompt on the next API call? Is the user informed of the mode change?

3. SUBGOAL IDENTIFICATION

Grouping the questions above produced four subgoals directly parallel to the GQ(I)M subgoal derivation process.

Subgoal 1: Improve clinical quality of AI responses. This was derived from questions about system prompts, model behaviour, and therapeutic appropriateness. It was addressed by building nine specialist system prompts grounded in Beck's CBT model, Linehan's DBT modules, Worden's grief tasks, and trauma-informed care principles.

Subgoal 2: Improve system reliability and fault tolerance. This was derived from questions about server stability, API error handling, and configuration issues. It was addressed by implementing try/catch blocks on all API calls, descriptive startup diagnostics, and graceful frontend error toasts.

Subgoal 3: Improve usability and accessibility. This was derived from questions about interface complexity and user guidance. It was addressed by building starter prompt pills, approach cards, a therapy tool toolbar, and a mood check-in widget reducing the steps required to begin any therapeutic function to a single click.

Subgoal 4: Improve security and deployment portability. This was derived from questions about API key exposure and cross-platform compatibility. It was addressed by server-side key storage

in .env, .gitignore protection, and a Node.js architecture that runs identically on Windows, macOS, and Linux.

4. ENTITIES AND ATTRIBUTES

Following Step 4 of the GQ(I)M process, entities and their measurable attributes were identified.

Entity: AI Therapy Response. Attributes: clinical accuracy (grounded in published framework yes or no), response warmth (validated by prompt design), response length in tokens, presence of validation before advice (yes or no), presence of open-ended closing question (yes or no).

Entity: Session. Attributes: duration in seconds tracked by session timer, message count measured as the length of the conversation history array, therapy mode selected as a nominal category, mood score submitted as an ordinal value from 1 to 10.

Entity: Server. Attributes: API key loaded (binary yes or no), response latency in milliseconds between request and reply, error rate measured as number of failed API calls per session, startup success as a boolean.

Entity: System Prompt. Attributes: length in characters, number of therapeutic techniques referenced, clinical framework cited (binary per framework), specificity of crisis protocol instructions (present or absent).

5. MEASUREMENT GOALS FORMALISED

Using the GQ(I)M measurement goal template, three formal measurement goals were produced.

Measurement Goal 1: Therapeutic Response Quality. Object of Interest: AI-generated therapy responses. Purpose: Evaluate the clinical appropriateness of responses in order to improve therapeutic accuracy. Perspective: Examine the presence of validation, reflective listening, psychoeducation, and one therapeutic technique per response from the viewpoint of the clinical framework designer. Environment: Nine therapy modes, Llama 3.3 70B model, temperature 0.75, maximum 800 tokens per response.

Measurement Goal 2: System Reliability. Object of Interest: Node.js server and Groq API integration. Purpose: Evaluate fault tolerance and error recovery in order to ensure uninterrupted therapeutic sessions. Perspective: Examine error handling coverage, startup diagnostics, and API timeout behaviour from the viewpoint of the developer and end user. Environment: Single-file Node.js server, Groq API endpoint, Windows 10 deployment, npm dependencies: express, cors, dotenv, node-fetch.

Measurement Goal 3: User Accessibility. Object of Interest: Frontend interface and interaction design. Purpose: Evaluate ease of use in order to minimise barriers for users in emotional distress. Perspective: Examine time-to-first-interaction, number of steps required to access any therapy tool, and clarity of navigation from the viewpoint of a non-technical user in crisis. Environment: Single HTML file, no login required, browser-based, mobile responsive.

6. QUANTIFIABLE QUESTIONS AND INDICATORS

Following Step 6, quantifiable questions were derived from each measurement goal and indicators were designed to answer them.

From Measurement Goal 1:

Quantifiable Question: What percentage of AI responses include a validation statement before advice? Indicator: Manual review of sample responses across all nine therapy modes checking for phrases such as "It sounds like..." or "What I'm hearing is..." or equivalent validation language.

Quantifiable Question: Does the system produce clinically distinct responses for each of the nine therapy modes for the same input? Indicator: Submit identical user message across all nine modes and compare response content. Distinct clinical technique per mode indicates correct system prompt differentiation.

From Measurement Goal 2:

Quantifiable Question: Does the server return a structured error message when the API key is missing rather than crashing? Indicator: Start server without .env key and observe whether the terminal displays a warning message and the server continues running. This is evaluated as a binary pass or fail.

Quantifiable Question: Does the frontend recover gracefully from an API timeout? Indicator: Simulate network failure during a session and observe whether a toast notification appears and conversation history is preserved. This is evaluated as a binary pass or fail.

From Measurement Goal 3:

Quantifiable Question: How many clicks does it take a new user to begin a therapy session? Indicator: Count interactions from page load to first AI response using the starter pill path with a target of one click and the manual typing path with a target of two actions being type and send.

Quantifiable Question: How long before a first-time user understands the system's purpose? Indicator: The welcome screen displays a purpose statement, approach cards, and starter prompts before any interaction. Time-to-comprehension is estimated as immediate within five seconds of page load.

7. DATA ELEMENTS IDENTIFIED

The following data elements were identified as required to construct the indicators above.

Session duration is collected by the JavaScript session timer incrementing every second once conversation history is non-empty.

Mood score is collected via the 10-button mood widget, stored as an integer from 1 to 10, and appended to conversation history as a user message.

Therapy mode is collected as a string variable covering general, cbt, dbt, mindfulness, grief, trauma, anxiety, depression, and relationships. It is updated on tab click and passed to the server with every API request.

API response latency is measurable as the time between the fetch() call and data resolution in the frontend JavaScript, observable in the browser developer tools network tab.

Error state is collected as a boolean derived from the presence of data.error in the server JSON response, triggering the toast notification display.

Conversation history length is measured as the length of the history array in the frontend, representing total message exchanges in the session.

8. MEASURE DEFINITIONS (Step 8)

Following the GQ(I)M requirement that measures be precisely defined, the following definitions were established.

Mood Score. Name: User-reported mood intensity. Scale: Ordinal, integer values 1 through 10. Range: 1 (very low) to 10 (great). Precision: Whole numbers only. Assumptions: Self-reported by user at point of check-in. Does not claim to be a clinical diagnostic instrument. Maps to therapeutic response calibration where scores 1 to 3 trigger crisis-aware language, scores 4 to 6 trigger supportive exploratory language, and scores 7 to 10 trigger growth-focused language.

Session Duration. Name: Therapeutic engagement time. Scale: Ratio, measured in seconds. Range: 0 to unlimited. Precision: 1 second intervals. Assumptions: Timer begins counting only after the first message is sent meaning conversation history length is greater than zero. Does not measure quality of engagement, only duration.

Therapy Mode. Name: Active therapeutic modality. Scale: Nominal, nine named categories. Range: general, cbt, dbt, mindfulness, grief, trauma, anxiety, depression, relationships. Precision: Exact string match to system prompt key. Assumptions: Each mode maps to one and only one system prompt. Switching mode mid-session appends a mode-change notification to the chat and changes the system prompt for all subsequent API calls.

Analysis

At the start of development, no existing measurement infrastructure existed. The required data elements covering mood score, session duration, mode selection, and error state were all absent from any prior system and needed to be built from scratch.

Diagnosis

The mood score could be collected via a custom HTML widget rather than an external survey tool. Session duration could be collected using a JavaScript setInterval timer already available in the browser. Mode selection was naturally captured by the existing tab click event. Error state was derivable from the API response JSON without any additional instrumentation.

Action

Data collection was implemented directly in the frontend JavaScript. The mood widget appends the score to conversation history. The session timer increments a counter displayed in the header. The mode variable is updated on tab click and passed in every POST request body. Error state is evaluated from data.error in the fetch response handler.